Formal Specification of Entity-Preserving RDB2RDF View

October 16, 2026

This document presents the formalism for specifying entity-preserving RDB2RDF views. By restricting ourselves to this class of views, we can precisely identify the tuples relevant to a data source update w.r.t. an RDB2RDF view. An *RDB2RDF view* is a triple $W = (V, S, M)$, where:

- V is the vocabulary of the *target ontology*;
- S is the *source relational schema*; and
- M is a set of *mappings* between V and S, defined by *transformation rules* (TRs).

A view $W = (V, S, M)$ satisfies the *entity-preserving property* iff:

- the instances of the classes in V correspond to tuples in selected relations of S, which are called the *pivot relations* of W, so that distinct tuples from the same pivot relation generate distinct class instances;
- the values of datatype properties in V of these instances are given by (functions of) attribute values of the corresponding tuples, or of related tuples;
- the object properties in V correspond to relationships between tuples of the pivot relations of S.

Intuitively, a view W satisfies the entity-preserving property iff it preserves the base entities (objects) of the source database, rather than creating new entities from the existing ones.

Transformation Rule (TR) patterns that lead to the definition of relational to RDF mappings that guarantees that the RDF views satisfy the entity-preserving property by construction. Specifically, there are three TR patterns:

- *Class Transformation Rule* (CTR);
- *Object Property Transformation Rule* (OTR); and
- *Datatype Property Transformation Rule* (DTR), further subdivided into *Local-DTR* and *Path-DTR*.

Intuitively, a CTR ψ maps tuples of a relation R into RDF instances of a class C. It establishes a semantic correspondence between a pivot tuple r and an RDF resource x, such that each pivot tuple is associated with at most one RDF instance, and distinct pivot tuples generate distinct RDF resources. Thus, the mapping preserves the identity of relational entities in the RDB2RDF view.

A Local-DTR has no relational path in the body so that the datatype values are obtained directly from attribute values of the pivot tuples. A Path-DTR has a relational path in the body, and the datatype values are obtained from tuples reachable through the relational path from the pivot tuples.

There are also two types of OTRs. A *regular OTR* defines object properties by relating an RDF instance generated from a pivot tuple t to an RDF instance generated from a pivot tuple u reached from t through a relational path in the source schema. Hence, OTRs capture relationships between entities represented in the database. A *reification OTR* relates an RDF instance generated from a pivot tuple t to a URI obtained by reification of a literal l created from attribute values of a pivot tuple u reached from t through a relational path in the source schema.

Transformation rules use several predicates. The simplest ones are:

- $nonNull(v)$ holds iff value v is not null;
- $RDFLiteral(u, A, R, v)$ holds iff v is the literal representation of a value u of attribute A of R;
- $RDFIRI(u, o)$ holds iff the non-null value u is converted into the RDF IRI o (that is, o is just the literal u treated as a URI);
- $FK(r, s)$ holds iff a tuple r of R is related to tuple s of S by a foreign key of the form $FK(R{:}L, S{:}K)$.

The more complex predicates are as follows. Let R be the pivot relation of a transformation rule ψ, and r be a pivot tuple of R. A *URI template* for R is a string pattern p with place holders $\langle h_1, \ldots, h_n \rangle$ such that, when the place holders are appropriately instantiated, the resulting string becomes a URI. A URI s is associated with r via p iff the predicate

$$hasURI(p, \langle e_1, \ldots, e_n \rangle, s)$$

holds, where:

- $\langle e_1, \ldots, e_n \rangle$ is an ordered sequence of expressions, where each e_i is of the form:
  - $e_i = r.A$, where $r.A$ denotes an attribute of the pivot tuple;
  - $e_i = [FK(r, t)/t.A]$, where $[FK(r, t)/t.A]$ denotes the value of attribute A obtained after following the foreign key FK from the pivot tuple r to tuple t such that the foreign-key navigation must be functional from the pivot relation to the referenced relation.
- s is the result of instantiating each h_i by e_i in p.

To simplify the notation, a second predicate, called a *URI constructor*, is introduced by definition:

$$URI_R(r, s) \equiv hasURI(p, \langle e_1, \ldots, e_n \rangle, s)$$

As an example, the URIs associated with the pivot relation *company* is defined using the predicate $hasURI$("http://example.com/company/$\langle h_1 \rangle$",$\langle r.ein \rangle, s$) indicating that s is generated by replacing h_1 by $r.ein$ value in the template "http://example.com/company/$\langle h_1 \rangle$".

If the transformation rule is generated from an R2RML statement, the R2RML subject-map template specifies the complete URI template, and $\langle e_1, \ldots, e_n \rangle$ is the ordered sequence of expressions corresponding to the placeholders of the R2RML URI template. Furthermore, every placeholder for an `rr:template` is translated into exactly one pivot-rooted attribute expression, preserving the complete URI template, the placeholder order, and the relational origin of each value.